Balme Maxence

Rapport Projet Applications Réseaux

Manuel d'Utilisation du Projet :

Introduction :

Ce projet a pour but de créer un éditeur collaboratif, qui est tenu par un serveur sur lequel les clients peuvent se connecter et ainsi modifier, créer ou supprimer des lignes. Nous verrons à travers ce projet différentes manières de gérer cette relation Serveur-Client tout d'abord en assurant les opérations depuis l'application GUI pour les clients, puis de s'assurer que les clients reçoivent les informations et modifient les mêmes lignes donc qu'il n'y ai pas de divergence dans la version du document pour les clients. Ensuite on étudiera des manières d'obtenir une latence plus faible ou encore de gérer des potentielles pannes de serveur.

Structure du Projet :

Src/ : Client : les classes client

- Autoclient.java : Client de test, 20 opérations par client, 100 ms entre modifications et attend 2 secondes entre chaque notification pour pas saturer
- GuiClient : Application de test Client complété dans la tache 1
- ServerConnection : gère la connexion au serveur, demande au Dispatch en Push
- ClientController : Affichage du document dans la ListView
- ClientPushLauncher : lance le GuiClient

: Server : les classes server

- ServerMaster.java : port 12340. Le serveur Master diffuse aux serveurs Slave.
- ServerSlave.java : port 12347-8..., connectés et reçoivent des infos du serverMaster Ils gèrent les clients qui se connectent
- ServerDispatch.java : port 12350, Round Robin sur les Slaves, dirige les clients qui demandent un serveur
- ServerCentralFederate : port 12345. Accepte les connexions serveurs/clients
- ServerReplicated : Serveur décentralisé pour la tache 7
- ClientHandler : traite les requêtes d'un client connecté au serveur
- SharedDocument : Structure du document partagé

: Build.gradle

: peers.cfg

: test_convergences.sh

Avancement par Tâche :

Tâche 1 :

Objectif : Créer un Serveur centralisé avec communication Client/serveur

1/ 'SharedDocument' : Structure de données = List<String > pour représenter le document partagé /

2/ On complète GUIClient pour qu'il interagisse avec le serveur

3/ La mise à jour des lignes est en temps réel, seulement cela ne se charge que lorsqu'on réactualise la page. Les modifications ne se font que pour un seul à la fois si deux clients modifient.

4/On complète en ajoutant un temps de pause quand l'utilisateur édite afin de ne plus rencontrer ce problème

Tâche 2 :

Objectif : Serveur en Mode Push au lieu de Pull

5/ On crée ServerCentralPush qui intègre le mode Push dans sa relation Serveur-Client. Ainsi on crée aussi ClientHandlerPush qui notifie les clients de toute opération sur le document. On crée ServerConnectionPush pour gérer la connexion entre un clientPush et serveurPush qui pointe désormais sur le port 12345 de ServerCentralPush. On a aussi un ClientControllerPush qui intègre désormais un thread d'écoute pour les requêtes. Pour finir on fait GUIclientPush pointant sur un clientviewPush qu'on a créé.

6/ On a ici créé une classe AutoClient s'occupant de créer des clients avec en paramètres un nombre de modification à faire ainsi qu'un id de client. On crée un script test_convergence.sh pour tester s'il y a des différences entre clients. Avec 3 clients testés, on obtient 3 versions différentes. Ce test nous confirme qu'il y a au contraire une divergence des versions qu'on peut retrouver en lançant plusieurs clients en même temps. On applique ensuite des temps d'attente pour bien laisser les notifications arriver et les opérations se faire dans l'ordre.

Tâche 3 :

Objectif : Serveurs communiquent entre eux (fédération)

7/ Un serveur peut se comporter comme un client vis-à-vis d'un autre serveur, il suffit de créer un Socket vers le port de l'autre serveur et la relation entre les 2 serveurs fonctionne comme une relation Serveur-Client

8/ On crée ServerCentralFederate qui a desormais un attribut List<PrintWriter> peers pour les clients. Le ServerSocket accepte les connexions de clients et de serveur, on le met sur le port 12345. On crée ainsi ClientHandlerFederate avec la commande CONNECT permettant de connecter les serveurs entre eux. On intègre un booléen isPeer pour soit notifier les clients soit notifier les serveurs lors d'une opération.

Tâche 4 :

Objectif : Tester la fédération entre 3 Serveurs

9/ On crée une fédération entre 3 serveurs un Serveur A, un serveur B et un serveur C. On va connecter le serveur A au serveur B et le serveur B au serveur C. On utilise les commandes CONNECT localhost port dans un terminal dédié.

10/ En testant cette configuration avec que j'évoque dans la question précédente avec un client distinct dans chaque serveur, on remarque que si le client A modifié ou ajoute une ligne, le client B recoit bien cette modification seulement le client C ne recoit pas la modification qu'a reçu B. Ainsi les clients n'ont pas la même version finale du document. Si on connecte A vers B, B vers A etc... On risque de créer une boucle de modification et ce n'est pas ce qu'on souhaite. J'ai aussi remarqué que dans cette configuration si B n'est pas vide car A a effectué des changements sur le document alors B ne peut plus diffuser à C, même sur des lignes qu'il crée. Cela cause donc un réel problème de comptabilité.

Tâche 5 :

Objectif : Avoir une Architecture Master/Slave

12/ On crée ServerMaster et ServerSlave. Les ServerSlave vont se connecter au ServerMaster et peuvent recevoir des clients qui se connectent. On a une hiérarchie de la sorte :

Client1 → ServerSlave1 → ServerMaster ↔ ServerSlave2 ← Client2
(port 12347) (port 12340) (port 12348)

ServerMaster va diffuser à tous les ServerSlave les commandes, c'est le serveur orchestrateur

13/ On crée le document peers.cfg de type : master = localhost 12340. Ainsi quand on lance serverMaster désormais, on lit la ligne ou est écrit master dans peers.cfg avec un buffer. On attribue le port associé à notre variable port. Pareil pour ServerSlave en attribuant le port et le host de ServerMaster.

Tâche 6 :

Objectif : Optimiser Performance, Round Robin

14/ Le client a le Serveur par défaut, il n'y a pas de choix de Server pour l'instant, ils vont tous sur le même client automatiquement

15/ On crée la classe ServerDispatch qui recoit la commande GETSERVER et s'occupant d'une liste de serverSlave. Son rôle est de retourner un ServerSlave par Round Robin, ce qui garantit une répartition équitable entre tous les ServerSlave disponibles pour n clients. La liste des slaves est récupéré depuis peers.cfg.

16/ On s'occupe d'enregistrer la latence et le nombre d'opérations moyen par 2 paramètres : le nombre de clients et le nombre de serveur. Voir la partie résultats et courbe pour les résultats de cette expérimentation.

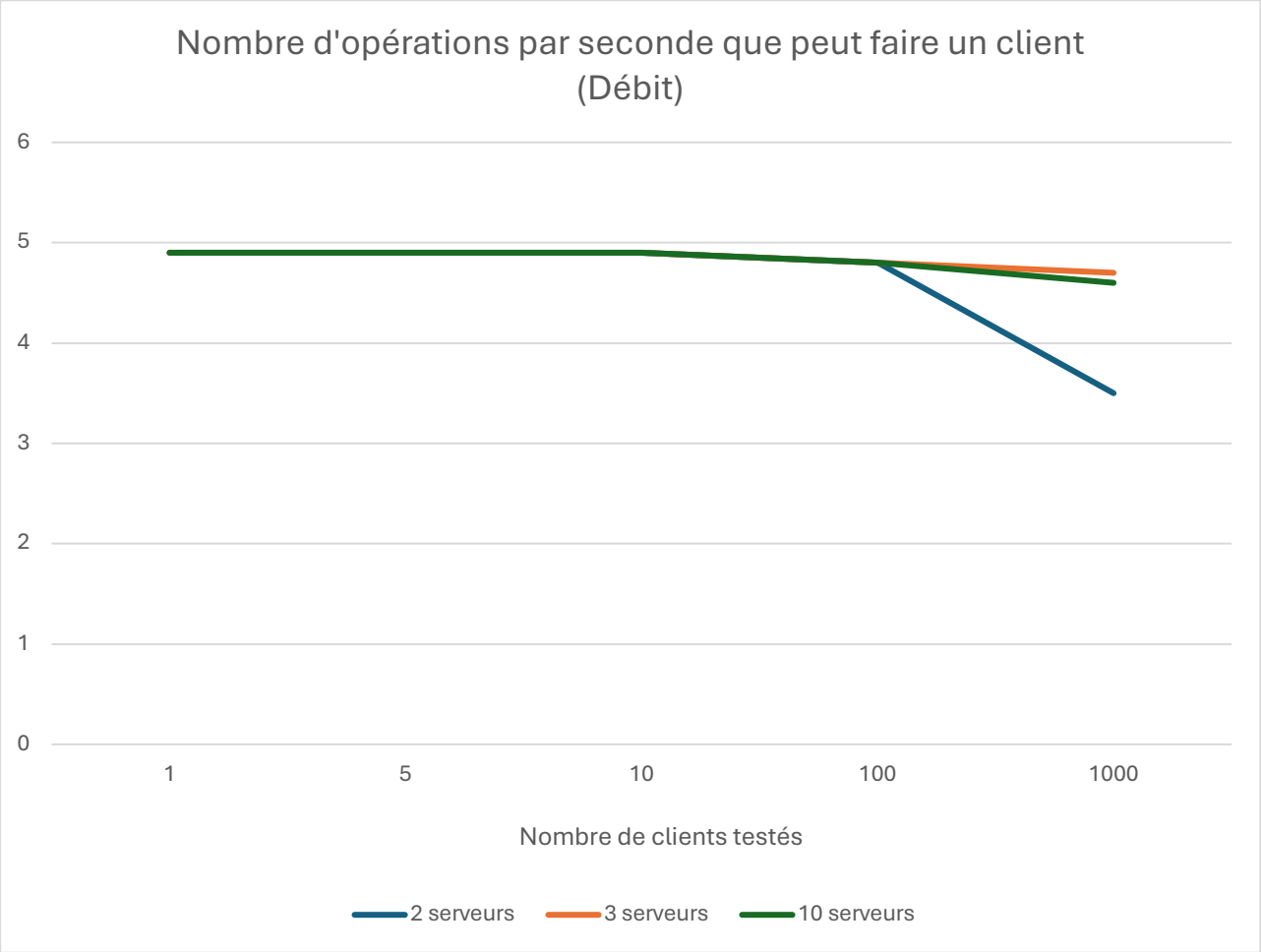
Tâche 7 :

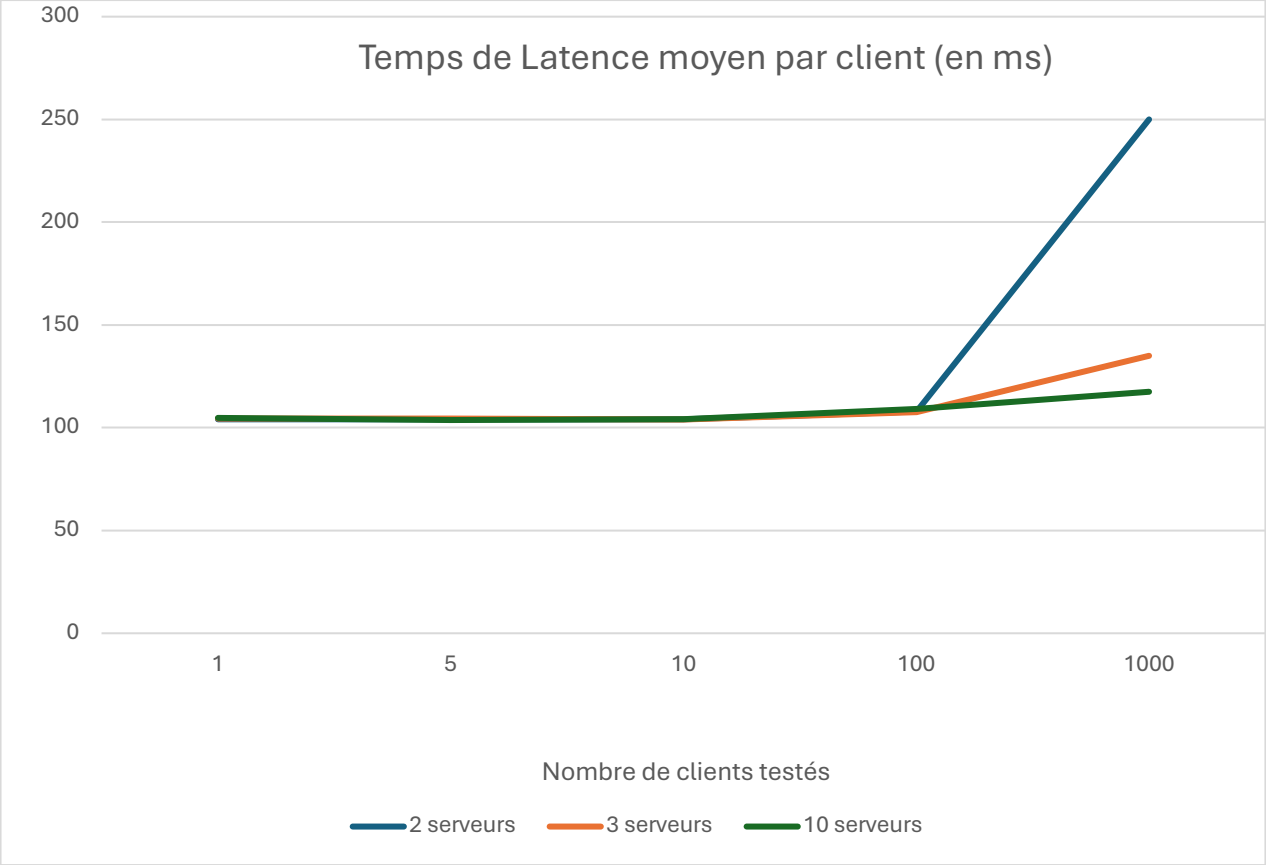
Objectif : Optimiser Remplacer Master/Slave par Pair à Pair

17/ On remarque que pendant la pause de 5 secondes, les modifications sont en attente donc augmentent beaucoup en latence. Le document ne se refresh plus pendant 5 secondes sur les applications. Les slaves ne peuvent plus propager leurs infos.

18/ On implémente ServerReplicated : on fait 3 serveurs qui se connectent entre eux automatiquement. Si un serveur lâche, les deux autres continueront d'être disponibles pour les clients qui s'y connectent. Les modifications se propagent entre les serveurs qui sont fonctionnels parmi les trois.

Résultats et Courbes :





Interprétations :

Les résultats sont dans l'ensemble très stables.

Débit : Stable à 4,9 op/s jusqu'à 100 clients. Grosse Chute quand on arrive à 1000 clients. L'augmentation de serveurs n'améliore pas le débit

Latence : La latence est stable à 104 ms jusqu'à 10 clients. Cela augmente légèrement pour 100 clients. La latence explose cependant pour 1000 clients, elle double carrément (250 ms)

En conclusion, le système marche bien jusqu'à 100 clients mais selon moi cela n'est pas adapté à un plus grand nombre de clients car on voit que la latence explose au-delà de 1000 clients.

Problèmes rencontrés :

- On a créé une classe ClientPushLauncher car il était impossible au début de lancer directement GUIClient utilisant JavaFx avec Java25
- Auto Refresh écrasait ce qu'on écrivait, j'ai arrangé en créant un booléen isEditing pour mettre en pause quand quelqu'un changeait une ligne
- Doublon de lignes au début quand on ajoutait une ligne simplement car on recevait le broadcast du serveur nous renvoyant une ligne